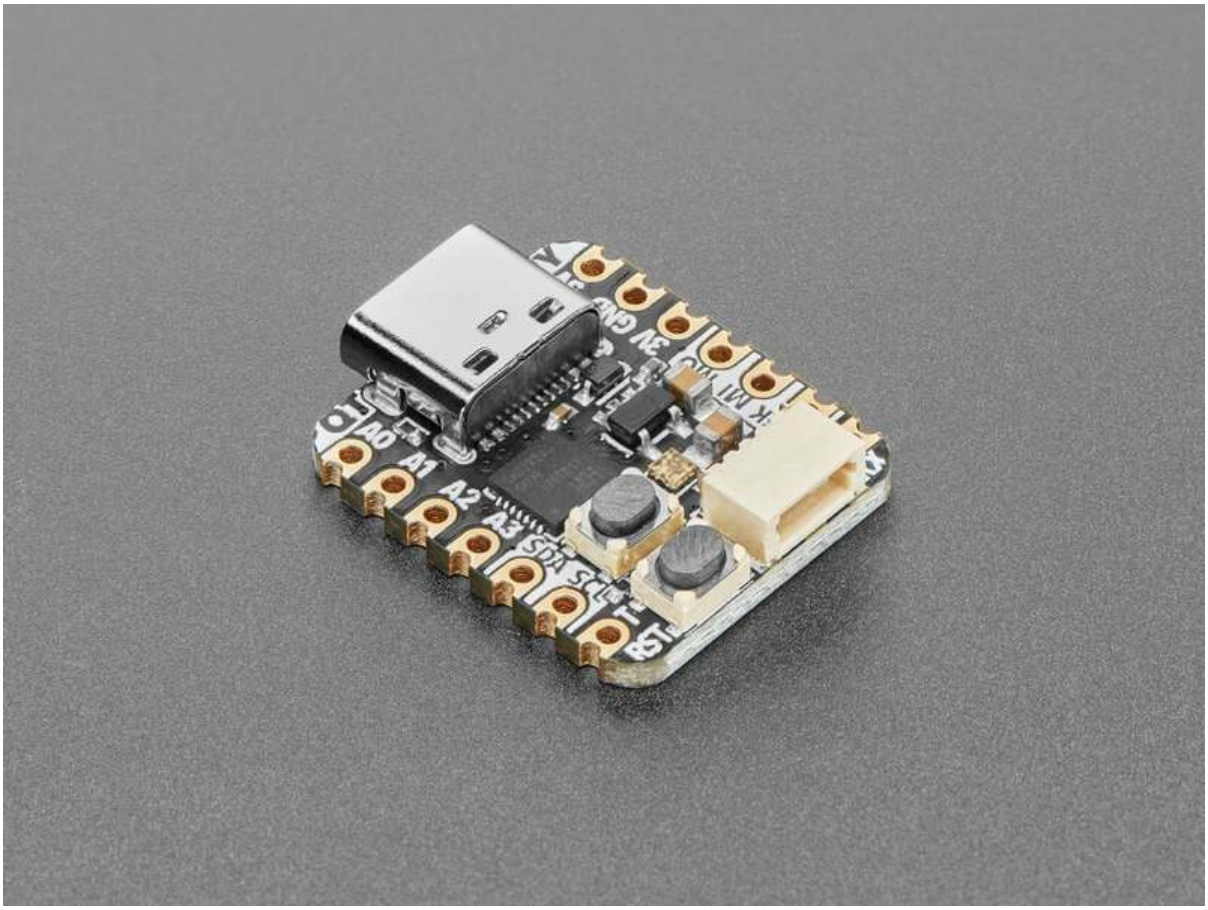
Adafruit QT Py CH32V203

Created by Liz Clark

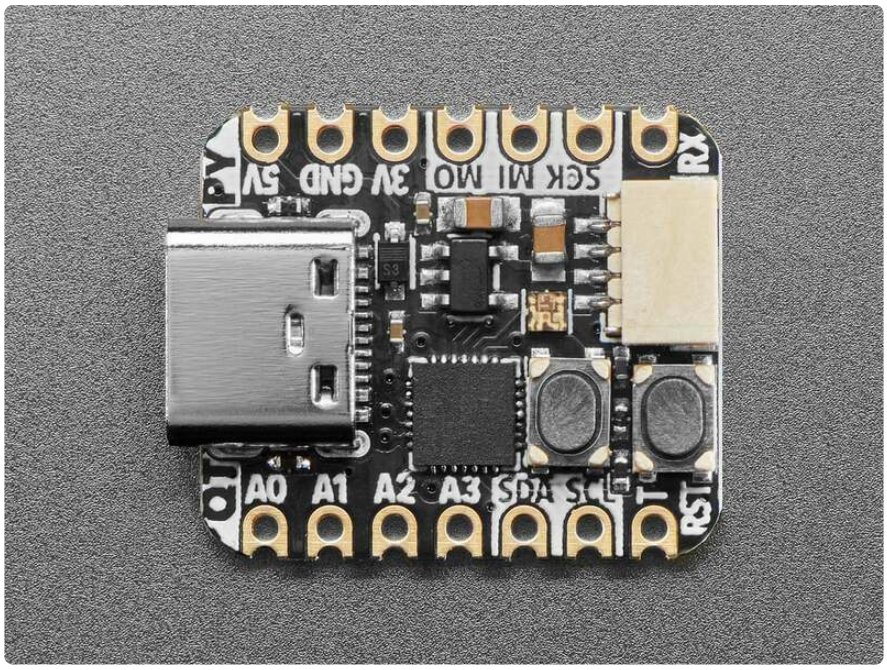


https://learn.adafruit.com/adafruit-qt-py-ch32v203

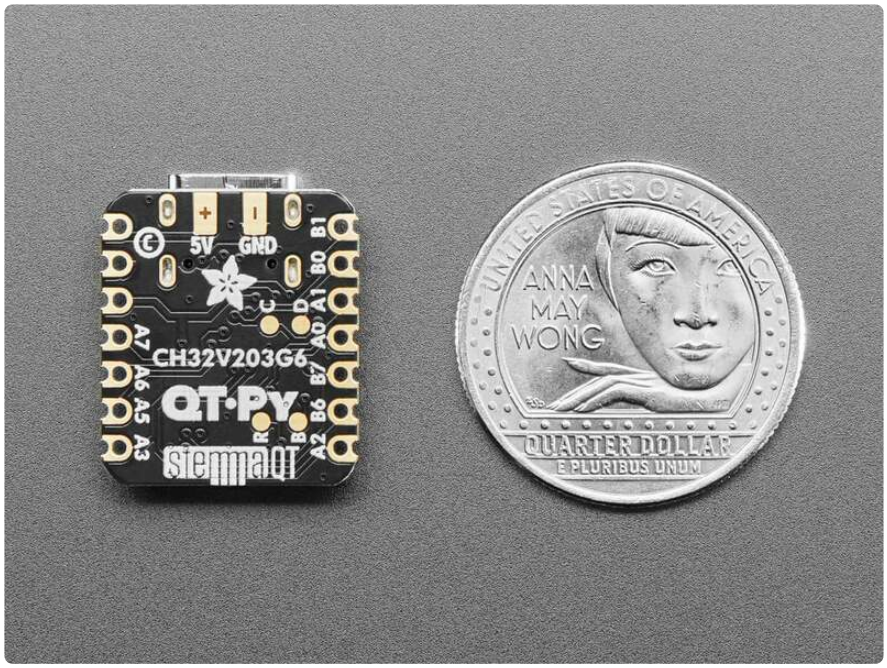
Last updated on 2024-10-07 07:51:16 PM EDT

Table of Contents

Overview

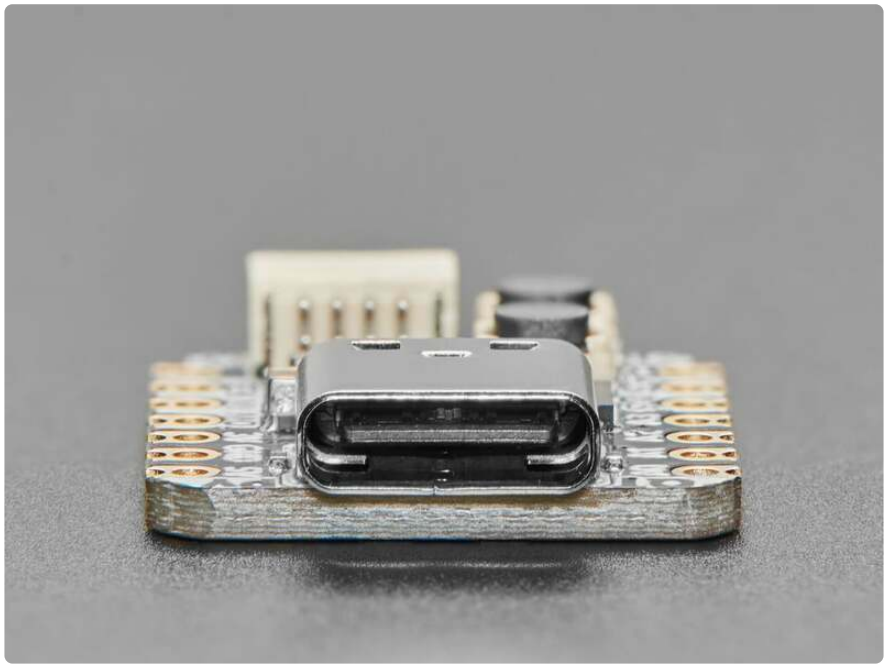


What a cutie pie! Or is it... a QT Py? This diminutive dev board features the powerful CH32V203 low-cost processor that's all the trend: based on RISC-V and cheaper than an 8-bit core! This little one is a great way to get started in the CH32x processor family, with everything you need to build many USB-based projects at an excellent price.

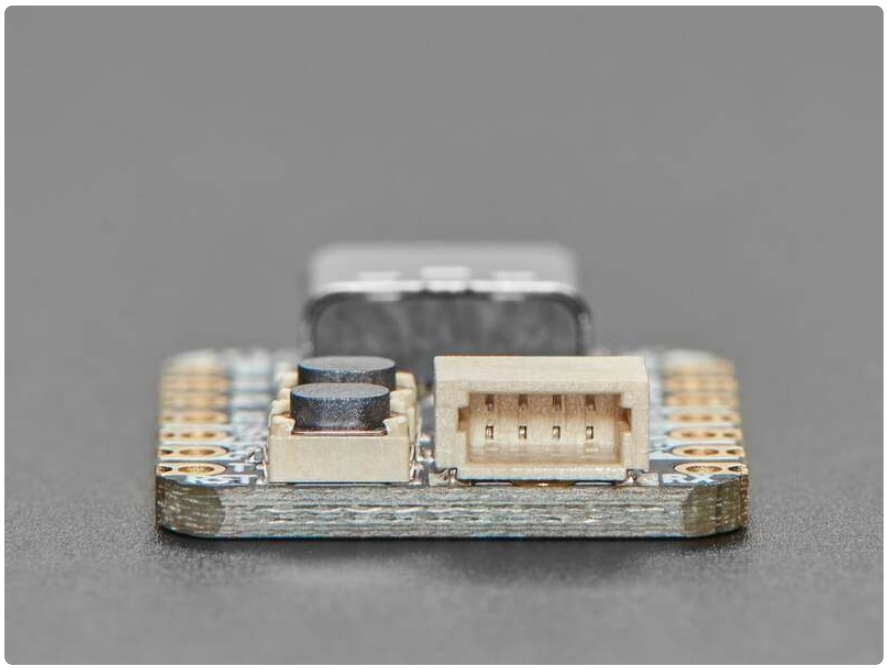


The **CH32V203G6** has a single 32-bit RISC-V core, running up to 144MHz, with 1-cycle multiply/divide. Inside is 10KB SRAM, 32KB single-cycle Flash as well as an additional 'external XIP' 224KB of Flash that can be used for program or data storage but is not

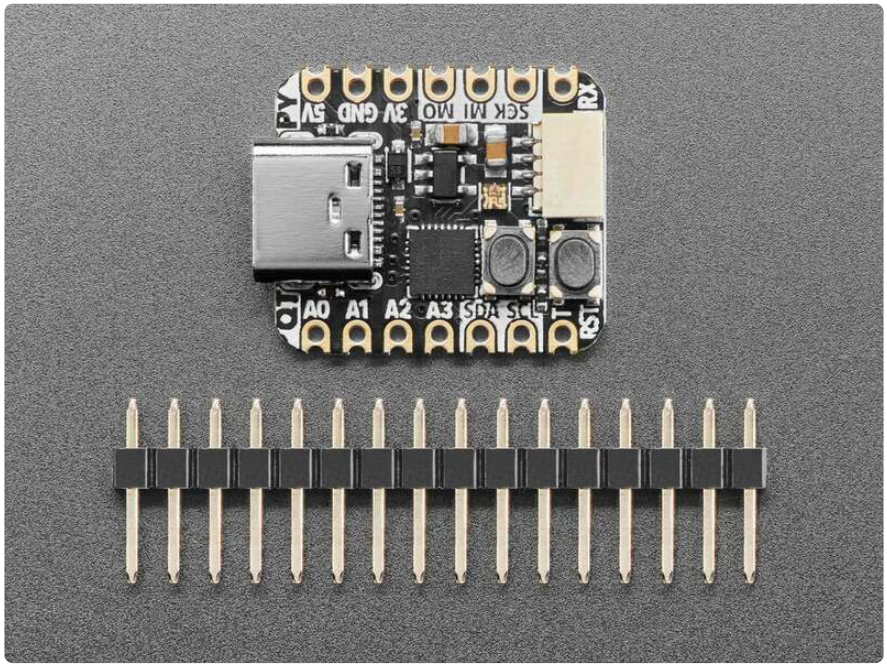
as fast as the 32KB. There's also extras you expect: ADC, timers, USB device, UART, I2C and SPI.



However, please note that the CH32 series is not supported nearly-as-well as ATmega, ESP32, ATSAMD, STM32, or RP2040 chips that have a strong company-led development community. WCH is very "it's a low cost chip, you figure it out" kind of company, and while there is some community-led development, it is still best used by folks comfortable with having to use Makefiles, clone git repositories, edit configuration files, etc. **It definitely does not run CircuitPython or Micropython, and Arduino support is very early.** It's definitely not good for beginners!



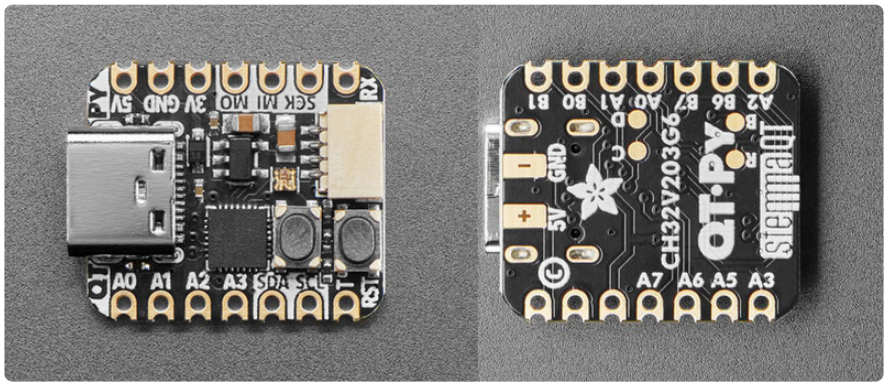
If you're interested in playing with the CH32V203, we've wrapped it up in a QT Py format. The pinout and shape is Seeed Xiao (https://adafru.it/NC3) compatible, with castellated pads so you can solder it flat to a PCB. It comes with our favorite connector - the STEMMA QT (https://adafru.it/HMB), a chainable I2C port that can be used with any of our STEMMA QT sensors and accessories (https://adafru.it/NmD). We also added an RGB NeoPixel and both a reset button and 'bootloader enter' button that allows you to upload code over USB without a WCH LINKE or similar SWD programmer.



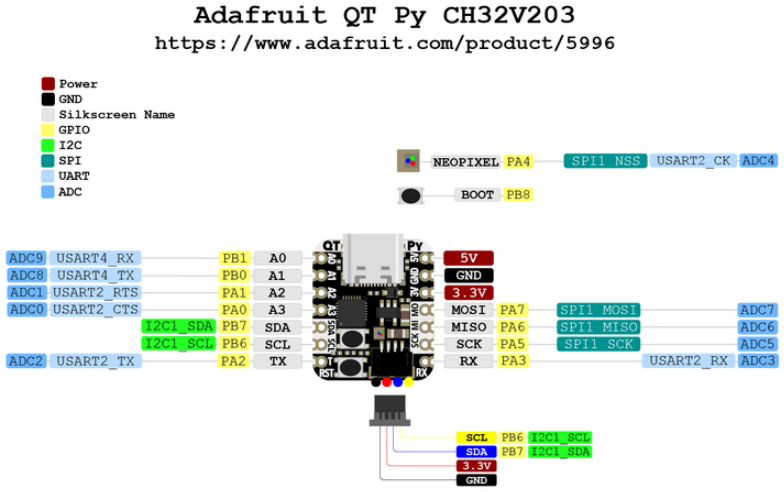
- Same size, form-factor, and pin-out as Seeed Xiao
- **USB Type C connector** - If you have only Micro B cables, this adapter will come in handy (http://adafru.it/4299)!
- **CH32V203G8 RISC-V** microcontroller core with 3.3V power/logic. Internal 144 MHz oscillator.
- **Native USB Device** - We do have USB CDC support via TinyUSB but its pretty chunky (https://adafru.it/1a6O) - upload can also be done via USB
- **Built in RGB NeoPixel LED**
- **10 GPIO pins:**
    - ADC input on all GPIOs
    - I2C port with STEMMA QT plug-n-play connector
    - Hardware UART
    - Hardware SPI
- 3.3V regulator with **600mA peak output** (https://adafru.it/NC4)
- **Reset switch and bootloader switch** for starting your project code over or entering USB ROM bootloader mode
- SWD data/clock pads on the bottom for advanced debugging usage

- Really really small

Pinouts

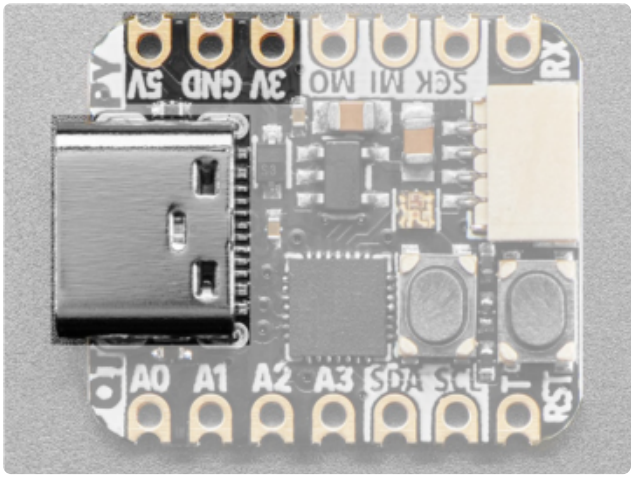


This QT Py does not run CircuitPython or MicroPython and Arduino support is very early.



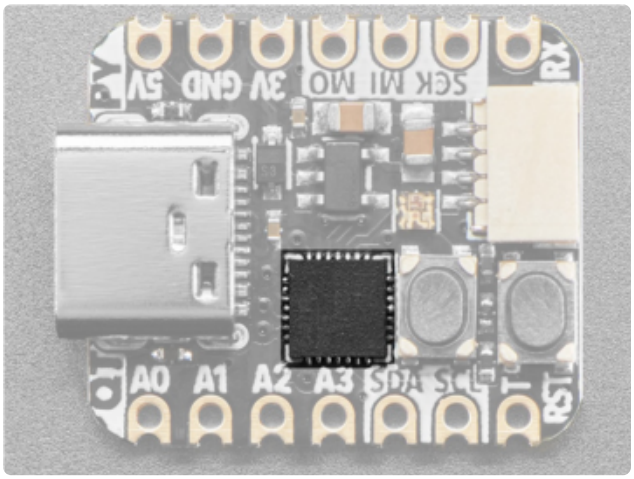
PrettyPins PDF on GitHub (https://adafru.it/1a6P).

Power



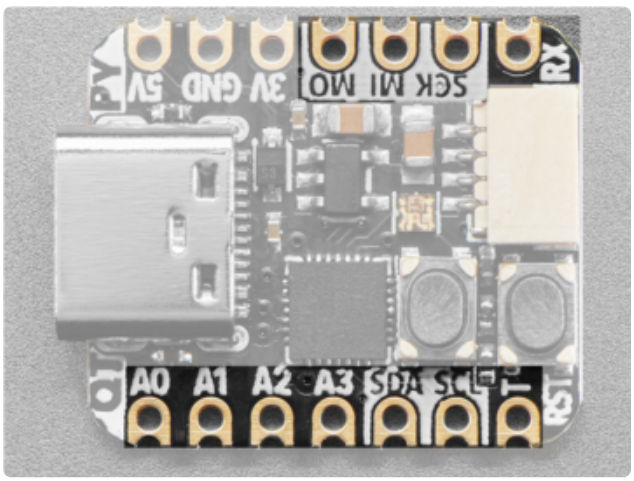
USB-C port - This is used for both powering and programming the board. You can power it with any USB C cable.
3V - This pin is the output from the 3.3V regulator (https://adafru.it/NC4), it can supply 600mA peak.
GND - This is the common ground for all power and logic.
5V - This is 5V out from the USB port.

CH32V203 Chip



The **CH32V203G6** has a single 32-bit RISC-V core, running up to 144MHz, with 1-cycle multiply/divide. Inside is 10KB SRAM, 32KB single-cycle Flash as well as an additional 'external XIP' 224KB of Flash that can be used for program or data storage but it's not as fast as the 32KB. There are also extras you expect: ADC, timers, USB device, UART, I2C and SPI.

Logic Pins



There are ten GPIO pins broken out to pins. There is hardware I2C, UART, and SPI. All of the GPIO have an ADC input.

I2C

- **SCL** - This is the I2C clock pin. There is no pull-up on this pin, so for I2C please add an external pull-up if the breakout doesn't have one already. It's connected to PB6.
- **SDA** - This is the I2C data pin. There is no pull-up on this pin, so for I2C please add an external pull-up if the breakout doesn't have one already. It's connected to PB7.

These pins are also connected to the STEMMA QT port.

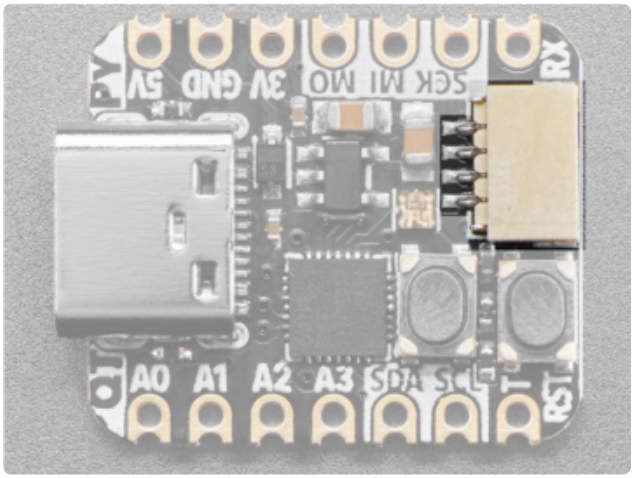
UART

- **RX** - This is the UART receive pin. Connect to TX (transmit) pin on your sensor or breakout. It's connected to PA3.
- **TX** - This is the UART transmit pin. Connect to RX (receive) pin on your sensor or breakout. It's connected to PA2.

SPI

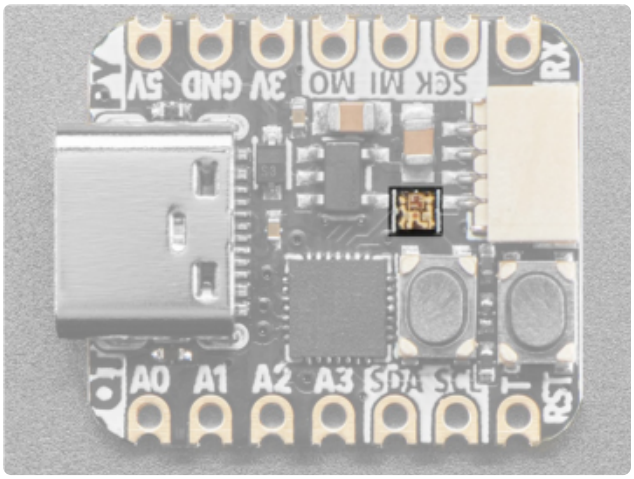
- **SCK** - This is the SPI clock pin. It's connected to PA5.
- **MI** - This is the SPI **M**icrocontroller **I**n / **S**ensor **O**ut pin. It's connected to PA6.
- **MO** - This is the SPI **M**icrocontroller **O**ut / **S**ensor **I**n pin. It's connected to PA7.

STEMMA QT Connector



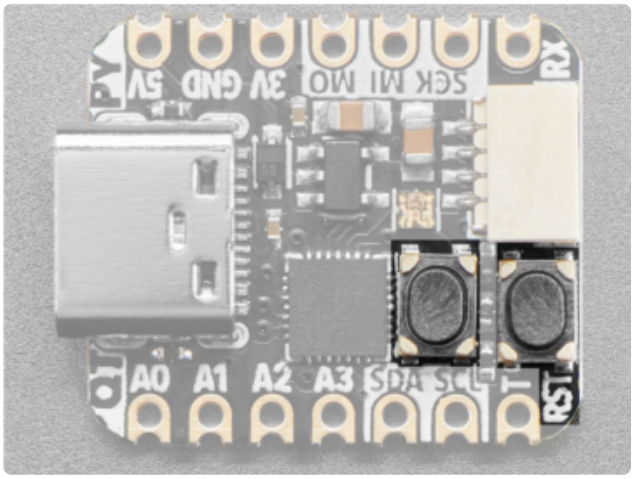
This **JST SH 4-pin** STEMMA QT **(https://adafru.it/Ft4) connector** is located at the back of the board. It allows you to connect to various breakouts and sensors with **STEMMA QT** connectors (https://adafru.it/Qgf) or to other things using assorted associated accessories (https://adafru.it/Ft6). It works great with any STEMMA QT or Qwiic sensor/device. You can also use it with Grove I2C devices thanks to this handy cable (http://adafru.it/4528).

NeoPixel LED



Next to the **BOOT** button, in the center of the board, is the **RGB NeoPixel LED**. This addressable LED can be controlled with code. It is connected to PA4.

Buttons



Reset button - This button restarts the board and helps enter the bootloader. You can click it once to reset the board without unplugging the USB cable or battery.
BOOT button - This button is connected to PB8/BOOT0. To enter bootloader mode, disconnect the QT Py from USB power. Hold down the **BOOT** button and reconnect USB power.

Arduino IDE Setup

You've seen the warnings that you definitely can't use this QT Py with CircuitPython or MicroPython. What you can do though is use the arduino_core_ch32 board support package (https://adafru.it/1a6Q) to write code in the Arduino IDE.

Support for the CH32V203 in the Arduino IDE is very new and Adafruit has contributed some changes to make interfacing with the QT Py easier. As a result, the steps on this page are more involved than what you may be used to when setting up the Arduino IDE for a new development board.

Install Arduino IDE

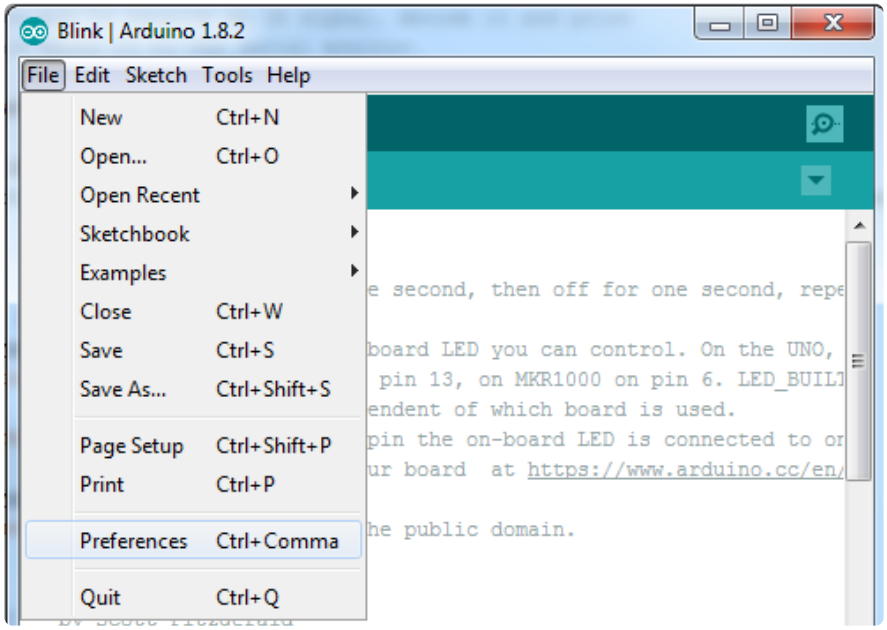
The first thing you will need to do is to download the latest release of the Arduino IDE. You will need to be using **version 1.8** or higher for this guide.

Arduino IDE Download

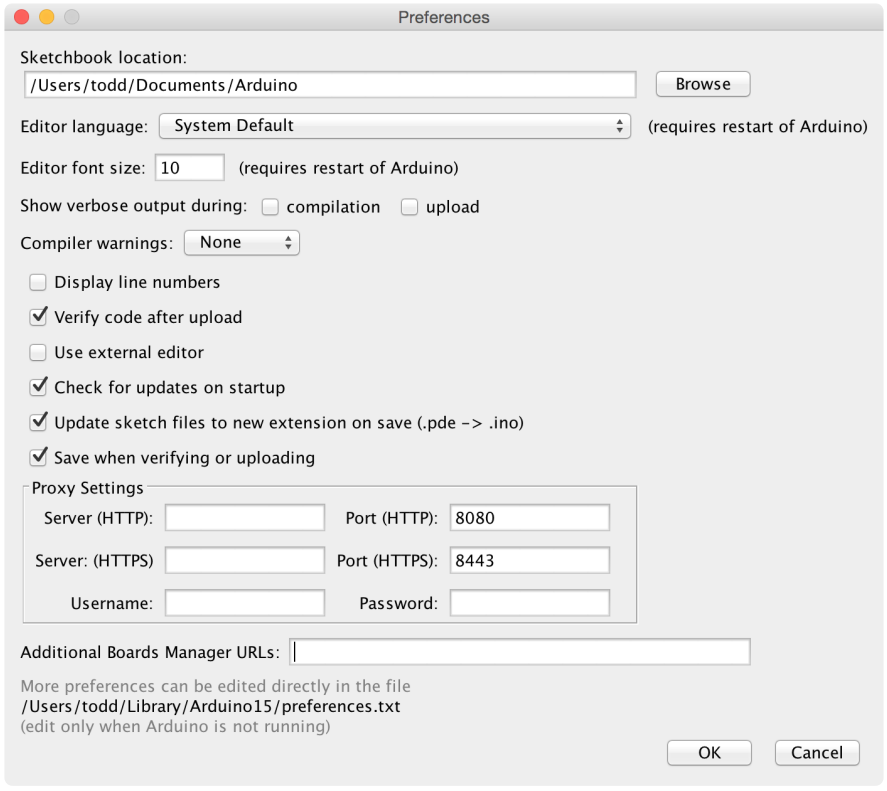
https://adafru.it/f1P

Install the arduino_core_ch32 Board Support Package

After you have downloaded and installed **the latest version of Arduino IDE**, you will need to start the IDE and navigate to the **Preferences** menu. You can access it from the **File** menu in Windows or Linux, or the **Arduino** menu on OS X.



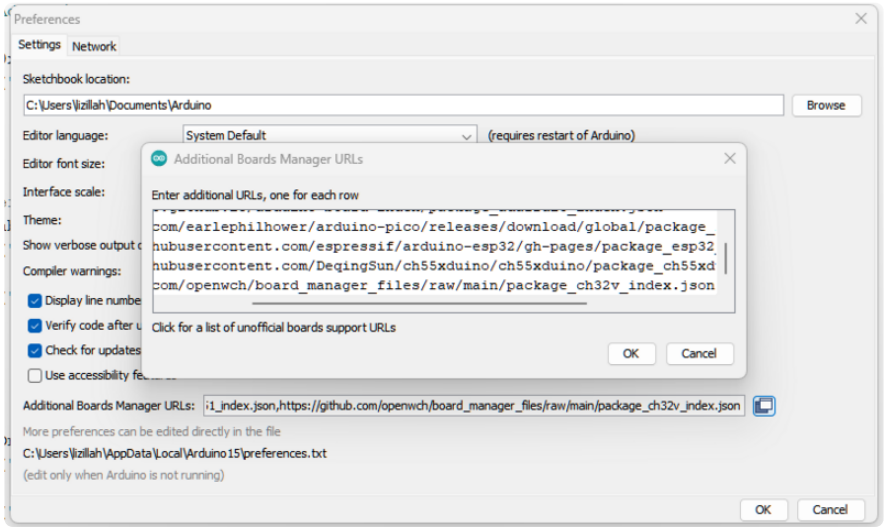
A dialog will pop up just like the one shown below.



We will be adding a URL to the new **Additional Boards Manager URLs** option. The list of URLs is comma separated, and you will only have to add each URL once. New Adafruit boards and updates to existing boards will automatically be picked up by the Board Manager each time it is opened. The URLs point to index files that the Board Manager uses to build the list of available & installed boards.

To find the most up to date list of URLs you can add, you can visit the list of third party board URLs on the Arduino IDE wiki (https://adafru.it/f7U). We will only need to add one URL to the IDE in this example, but **you can add multiple URLS by separating them with commas**. Copy and paste the link below into the **Additional Boards Manager URLs** option in the Arduino IDE preferences.

```
https://github.com/openwch/board_manager_files/raw/main/package_ch32v_index.json
```



If you have multiple boards you want to support, say ESP8266 and Adafruit, have both URLs in the text box separated by a comma (,)

Once done click **OK** to save the new preference settings.

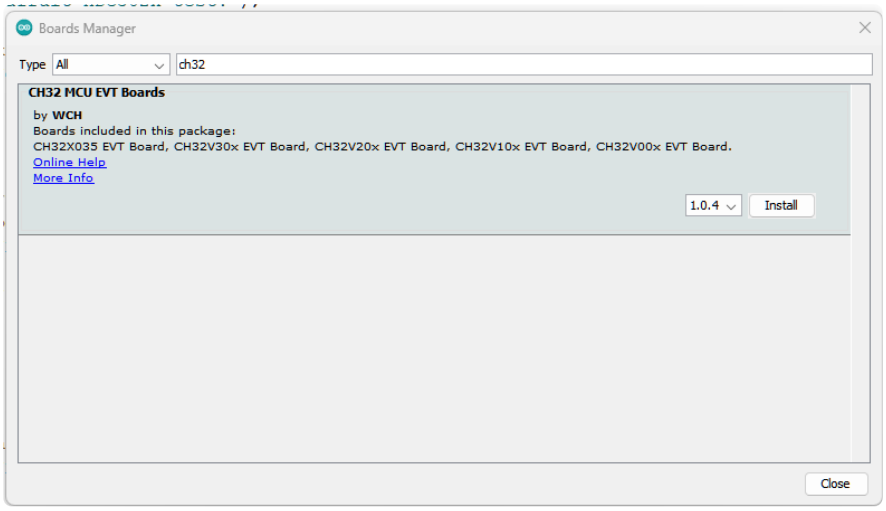
Installation Prerequisites

Before you install the board support package via the Arduino IDE, make sure to check the toolchain prerequisites for your operating system (https://adafru.it/1a6R) in the board support package repository. It's very important that you follow the steps listed in the README before installing the board support package in the Arduino IDE.

Follow the steps listed in the GitHub README for your operating system before installing the board support package in the Arduino IDE.

Install with the Board Manager

The next step is to actually install the Board Support Package (BSP). Go to the **Tools → Board → Boards Manager** submenu. A dialog should come up with various BSPs. Search for **ch32**.

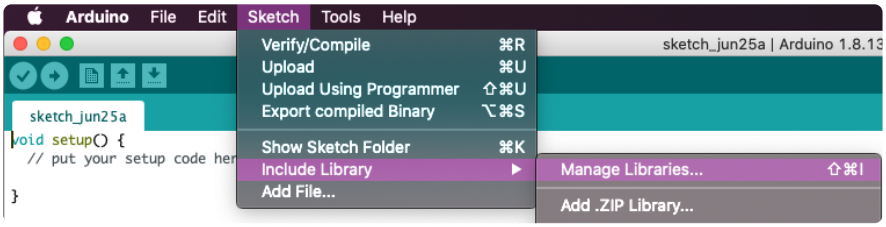


Click the **Install** button and wait for it to finish. Once it is finished, you can close the dialog. This takes care of installing all of the backend dependencies and toolchain for the WCH chips.

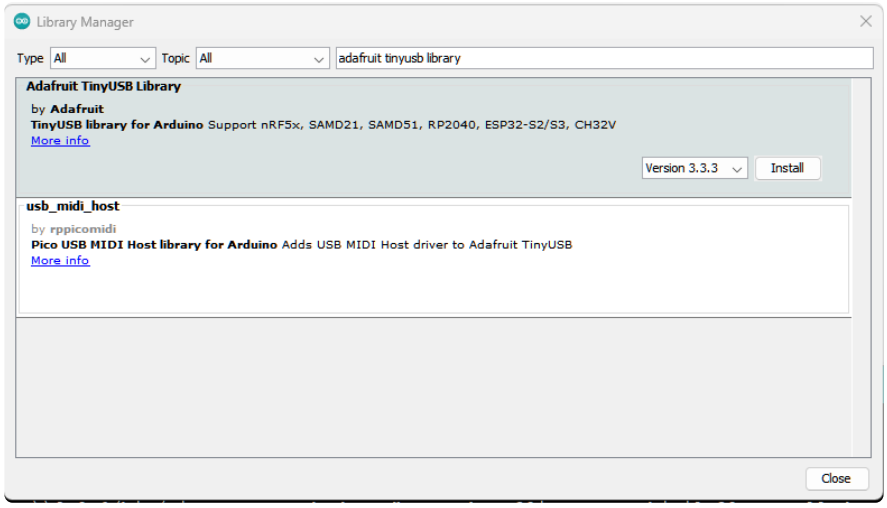
Install the Adafruit TinyUSB Arduino Library

A lot of effort and time has been put into updating the TinyUSB Arduino library to support the CH32V203 QT Py. These updates allow you to upload code easily to the QT Py over USB. You'll need to install version 3.3.3 (https://adafru.it/1a6S) or newer to use these changes.

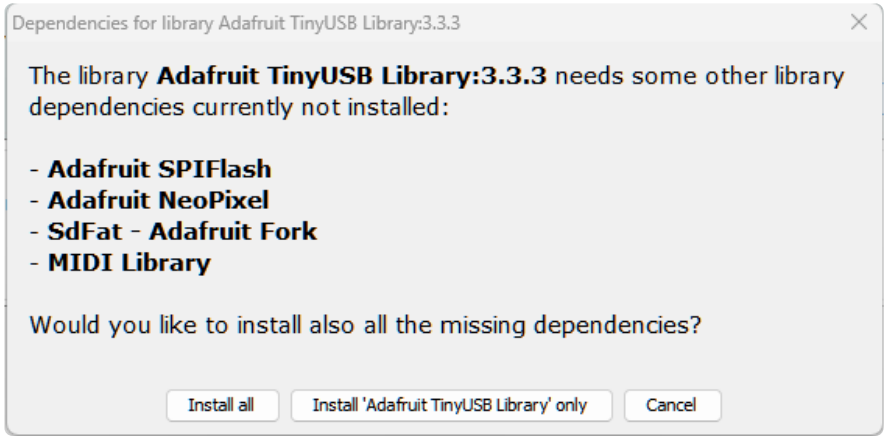
Version 3.3.3 or newer of the Arduino TinyUSB Library is required to properly interface with the QT Py



Click the **Manage Libraries ...** menu item, search for **Adafruit TinyUSB**, and select the **Adafruit TinyUSB** library:



If asked about dependencies, click "Install all".



If the "Dependencies" window does not come up, then you already have the dependencies installed.

Manually Install the arduino_core_ch32 Board Support Package

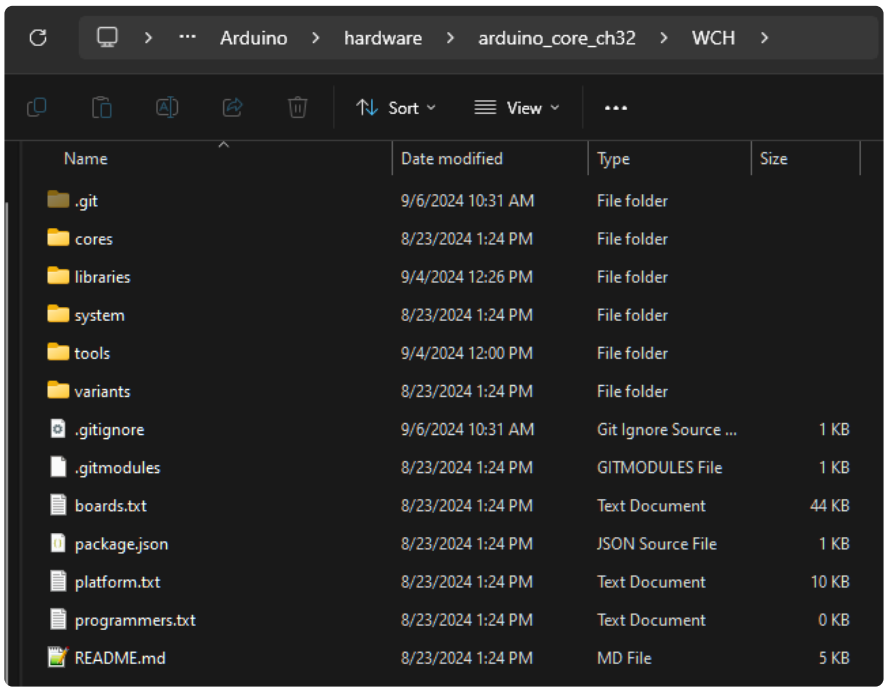
A pull request was recently merged to the CH32 board support package to add a board definition for the QT Py CH32V203. Until a new release is made, you'll want to manually install the board support package to get these newest features. To do this, you'll need to **git clone** the repository into the **hardware** folder inside of your **Arduino** folder in your filesystem.

You can do this using the GitHub Desktop app or by using `git` in the terminal:

```
cd /path/to/Arduino/hardware
git clone https://github.com/openwch/arduino_core_ch32.git
```

There is one final step to have the manual installation of the board support package appear in the Arduino IDE. Navigate to the **arduino_core_ch32** folder inside of the **hardware** folder. You'll need to **create a new folder called WCH**. **Move the contents of the arduino_core_ch32** folder that you just cloned into that **WCH folder**.

Your new folder hierarchy should be **Arduino/hardware/arduino_core_ch32/WCH**



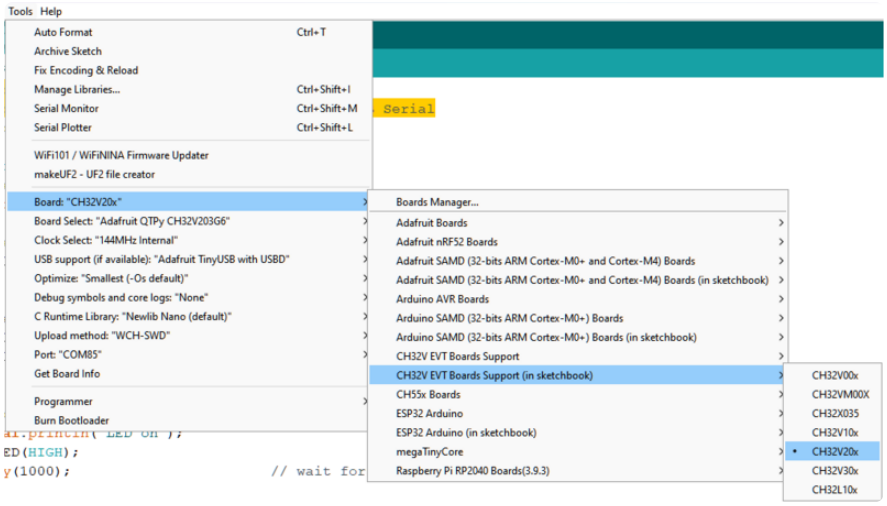
You may need to update the Adafruit TinyUSB submodule that came with the git clone by doing this:

```
cd /path/to/Arduino/hardware/arduino_core_ch23/WCH
git submodule update --init
```

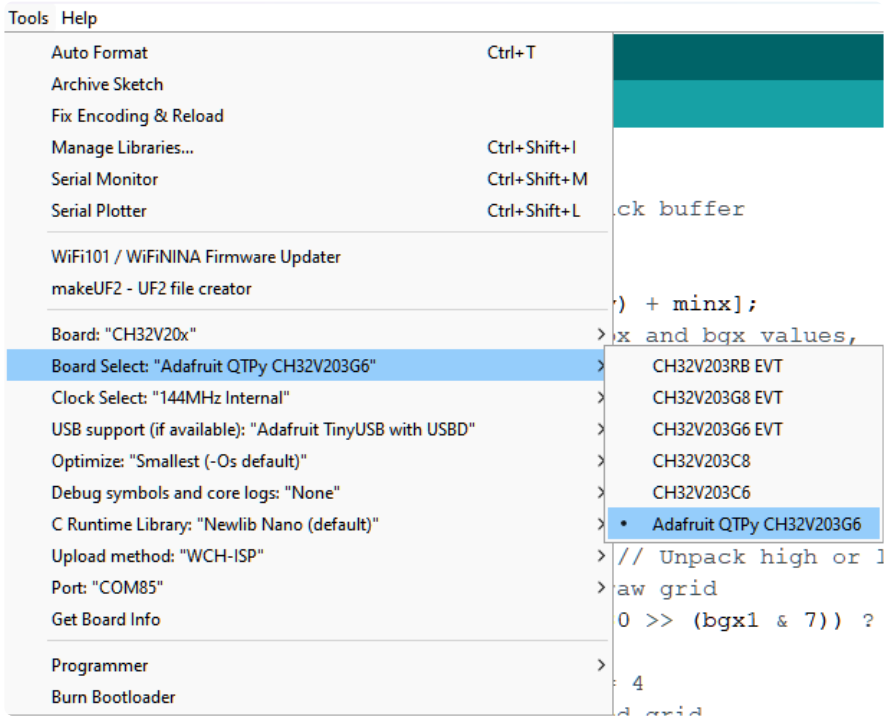
Restart the Arduino IDE to be sure that it grabs the forked board support package in the hardware folder.

Code Upload Options

In the **Tools → Board** submenu you should see **CH32V EVT Boards Support (in sketchbook)**. This is the manual installation that you just cloned into your **hardware** folder.



Under **Board**, select **CH32V20x**.



Under **Board Select**, select **Adafruit QTPy CH32V203G6**. Under **USB Support**, select **Adafruit TinyUSB with USBD**. These settings will work with the CH32V203 QT Py.

If you're not able to upload to the board, you may need to 'manually' put it into bootloader mode and use the WCHISP tool. Check this page https://learn.adafruit.com/adafruit-qt-py-ch32v203/bootloader-mode

Blink

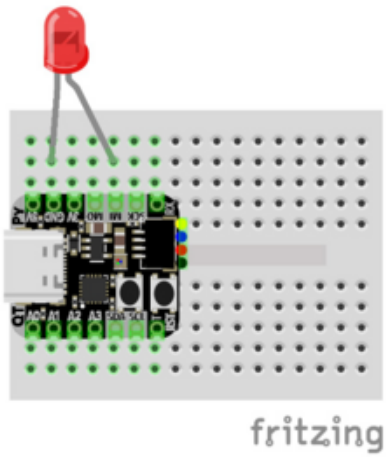
Blinking an LED is a great way to determine that you have your hardware and software ducks in a row. In this example, you'll breadboard an LED to the **MISO** pin (**PA6**), upload the example code and see your LED blink on and off every second.

Diffused 10mm LED Pack - 5 LEDs each in 5 Colors - 25 Pack

Need some chunky indicators? We are big fans of these diffused LEDs. They are fairly bright, so they can be seen in daytime, and from any angle. They go easily into a breadboard and...

https://www.adafruit.com/product/4204

Wiring



Board GND to **LED cathode (black wire)**

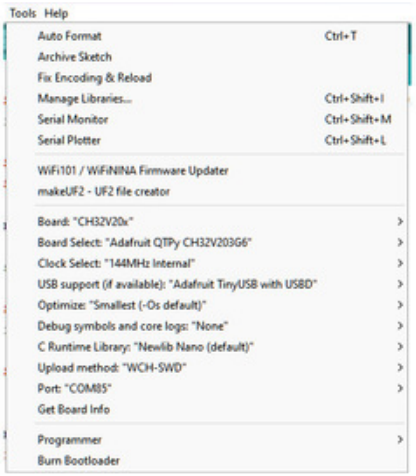
Board MISO to **LED anode (red wire)**

Blink Example

```
// SPDX-FileCopyrightText: 2024 Ha Thach for Adafruit Industries
//
// SPDX-License-Identifier: MIT

#include <Arduino.h>
#include <Adafruit_TinyUSB.h> // required for USB Serial

const int led = PA6;

void setup () {
  pinMode(led, OUTPUT);
}

void loop () {
  Serial.println("LED on");
  digitalWrite(led, HIGH);   // turn the LED on (HIGH is the voltage level)
```

```
  delay(1000);               // wait for a second

  Serial.println("LED off");
  digitalWrite(led, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);               // wait for a second
}
```

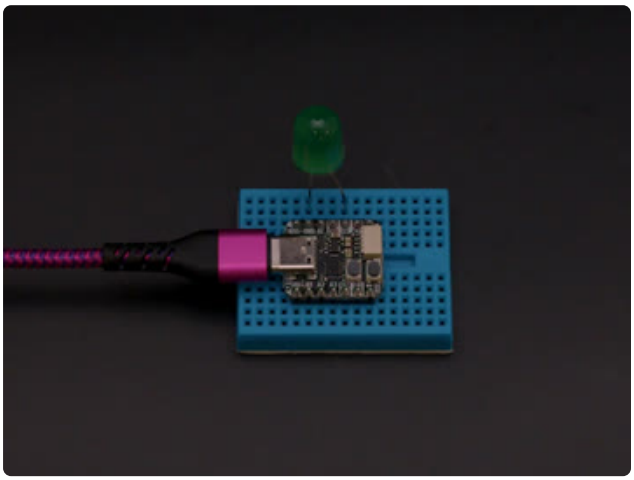
You'll notice that at the top of the code that the TinyUSB library is imported with `#include <Adafruit_TinyUSB.h>`. This will allow access to the USB serial port on the QT Py.



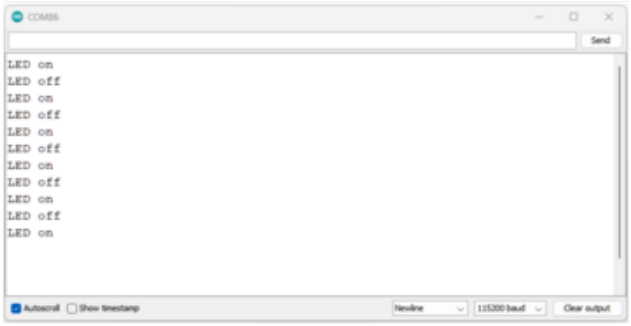
Confirm that your upload settings match the settings listed here under **Tools**:

Board: **CH32V20x**
Board Select: **Adafruit QTPy CH32V203G6**
USB support: **Adafruit TinyUSB with USBD**

For the port, select the COM port that matches your QT Py. It will not be labeled like you may be used to with other boards in the Arduino IDE.



Upload the sketch to your board. You should see the LED blink on and off every second.



If you open the Serial Monitor, you'll see `LED on` and `LED off` print out in unison with the blinking LED.

If you're not able to upload to the board, you may need to 'manually' put it into bootloader mode and use the WCHISP tool. Check this page https://learn.adafruit.com/adafruit-qt-py-ch32v203/bootloader-mode

---

I2C Scan Test

A lot of sensors, displays, and devices can connect over I2C. I2C is a 2-wire 'bus' that allows multiple devices to all connect on one set of pins so it's very convenient for wiring!

When using your board, you'll probably want to connect up I2C devices, and it can be a little tricky the first time. The best way to debug I2C is go through a checklist and then perform an I2C scan

Common I2C Connectivity Issues

- **Have you connected four wires (at a minimum) for each I2C device?** Power the device with whatever is the logic level of your microcontroller board (probably 3.3V), then a ground wire, and a SCL clock wire, and and a SDA data wire.
- **If you're using a STEMMA QT board - check if the power LED is lit.** It's usually a green LED to the left side of the board.
- **Does the STEMMA QT/I2C port have switchable power or pullups?** To reduce power, some boards have the ability to cut power to I2C devices or the pullup resistors. Check the documentation if you have to do something special to turn on the power or pullups.
- **If you are using a DIY I2C device, do you have pullup resistors?** Many boards do not have pullup resistors built in and they are required! We suggest any

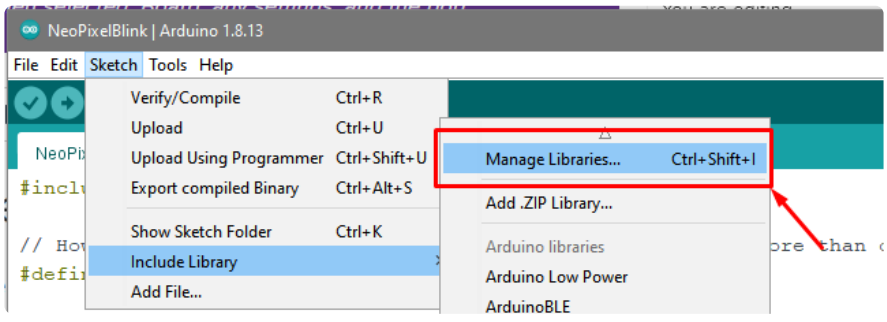
common 2.2K to 10K resistors. You'll need two: one each connects from SDA to positive power, and SCL to positive power. Again, positive power (a.k.a VCC, VDD or V+) is often 3.3V

- **Do you have an address collision?** You can only have one board per address. So you cannot, say, connect two AHT20's to one I2C port because they have the same address and will interfere. Check the sensor or documentation for the address. Sometimes there are ways to adjust the address.
- **Does your board have multiple I2C ports?** Historically, boards only came with one. But nowadays you can have two or even three! This can help solve the "hey, but what if I want two devices with the same address" problem: just put one on each bus.
- **Are you hot-plugging devices?** I2C does not support dynamic re-connection, you cannot connect and disconnect sensors as you please. They should all be connected on boot and not change. (Only exception is if you're using a hot-plug assistant but that'll cost you (http://adafru.it/5159)).
- **Are you keeping the total bus length reasonable?** I2C was designed for maybe 6" max length. We like to push that with plug-n-play cables, but really please keep them as short as possible! (Only exception is if you're using an active bus extender (http://adafru.it/4756)).

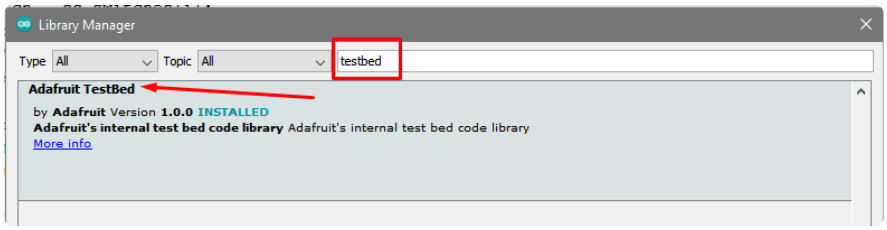
Perform an I2C scan!

Install TestBed Library

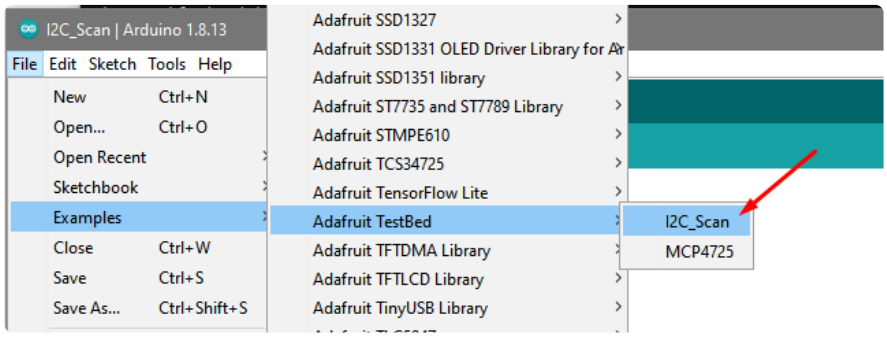
To scan I2C, the Adafruit TestBed library is used. This library and example just makes the scan a little easier to run because it takes care of some of the basics. You will need to add support by installing the library. Good news: it is very easy to do it. Go to the **Arduino Library Manager.**



Search for **TestBed** and install the **Adafruit TestBed** library



Now open up the I2C Scan example



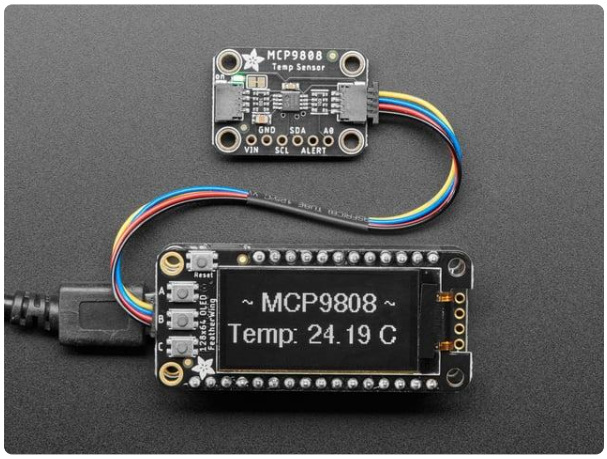

```
// SPDX-FileCopyrightText: 2023 Carter Nelson for Adafruit Industries
//
// SPDX-License-Identifier: MIT
// ------------------------------------
// i2c_scanner
//
// Modified from https://playground.arduino.cc/Main/I2cScanner/
// ------------------------------------

#include <Wire.h>

// Set I2C bus to use: Wire, Wire1, etc.
#define WIRE Wire

void setup() {
  WIRE.begin();

  Serial.begin(9600);
  while (!Serial)
     delay(10);
  Serial.println("\nI2C Scanner");
}


void loop() {
  byte error, address;
  int nDevices;

  Serial.println("Scanning...");

  nDevices = 0;
  for(address = 1; address < 127; address++ )
  {
    // The i2c_scanner uses the return value of
    // the Write.endTransmisstion to see if
    // a device did acknowledge to the address.
    WIRE.beginTransmission(address);
    error = WIRE.endTransmission();

    if (error == 0)
    {
      Serial.print("I2C device found at address 0x");
      if (address<16)
```

```
      Serial.print("0");
    Serial.print(address,HEX);
    Serial.println("  !");

    nDevices++;
  }
  else if (error==4)
  {
    Serial.print("Unknown error at address 0x");
    if (address<16)
      Serial.print("0");
    Serial.println(address,HEX);
  }
 }
 if (nDevices == 0)
   Serial.println("No I2C devices found\n");
 else
   Serial.println("done\n");

 delay(5000);           // wait 5 seconds for next scan
}
```

Wire up I2C device

While the examples here will be using the Adafruit MCP9808 (http://adafru.it/5027), a high accuracy temperature sensor, the overall process is the same for just about any I2C sensor or device.

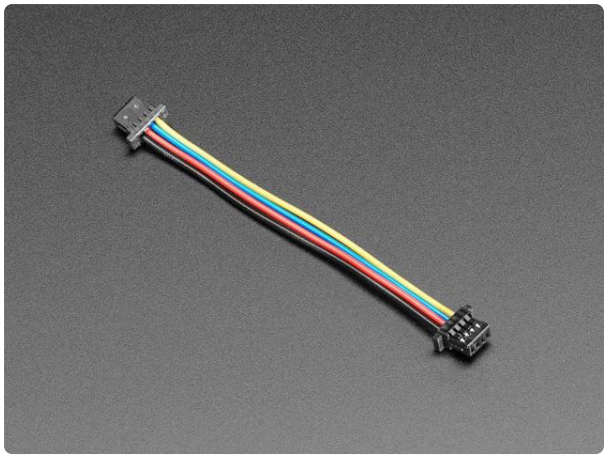
The first thing you'll want to do is get the sensor connected so your board has I2C to talk to.



Adafruit MCP9808 High Accuracy I2C Temperature Sensor Breakout

The MCP9808 digital temperature sensor is one of the more accurate/precise we've ever seen, with a typical accuracy of ±0.25°C over the sensor's -40°C to...

https://www.adafruit.com/product/5027



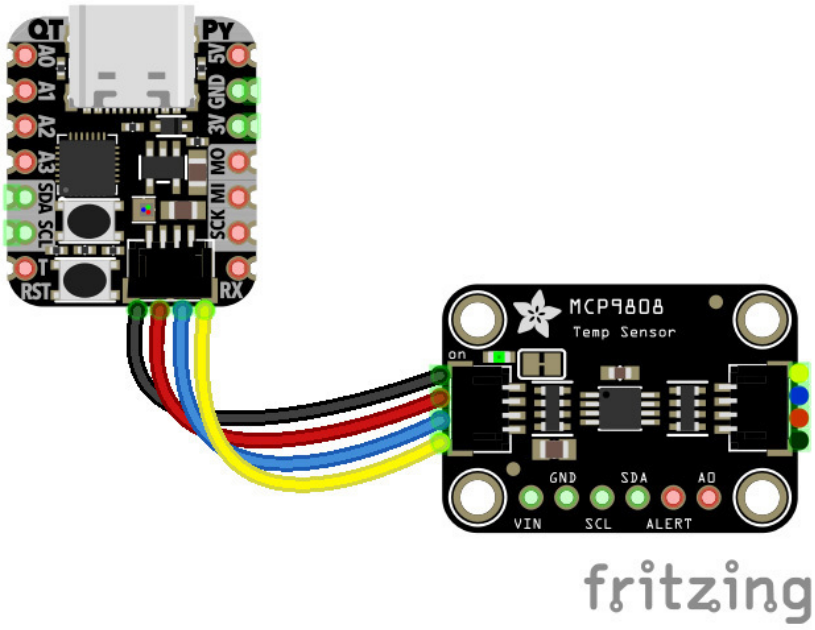
STEMMA QT / Qwiic JST SH 4-Pin Cable - 50mm Long

This 4-wire cable is 50mm / 1.9" long and fitted with JST SH female 4-pin connectors on both ends. Compared with the chunkier JST PH these are 1mm pitch instead of 2mm, but...

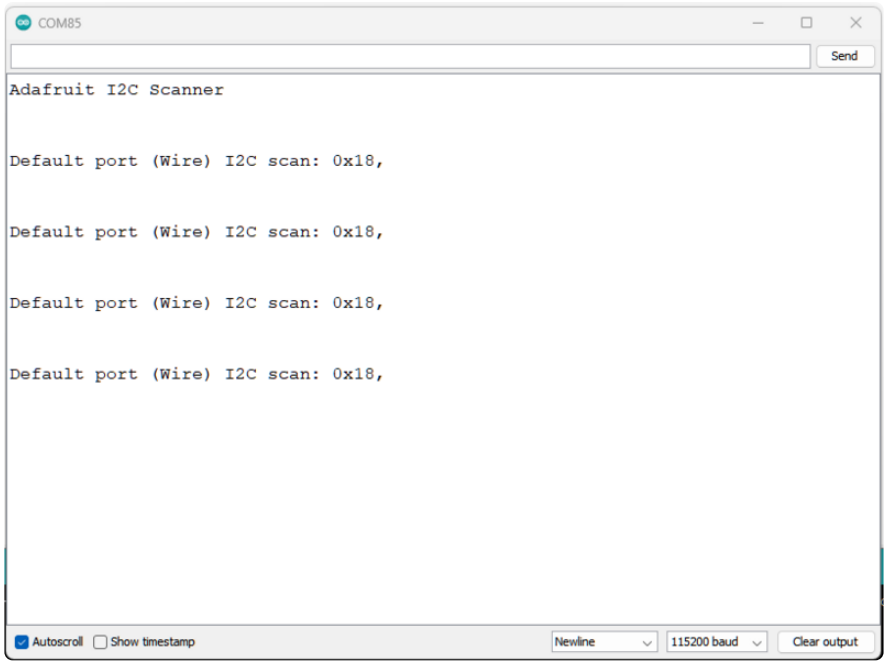
https://www.adafruit.com/product/4399

Wiring the MCP9808

The MCP9808 comes with a STEMMA QT connector, which makes wiring it up quite simple and solder-free.

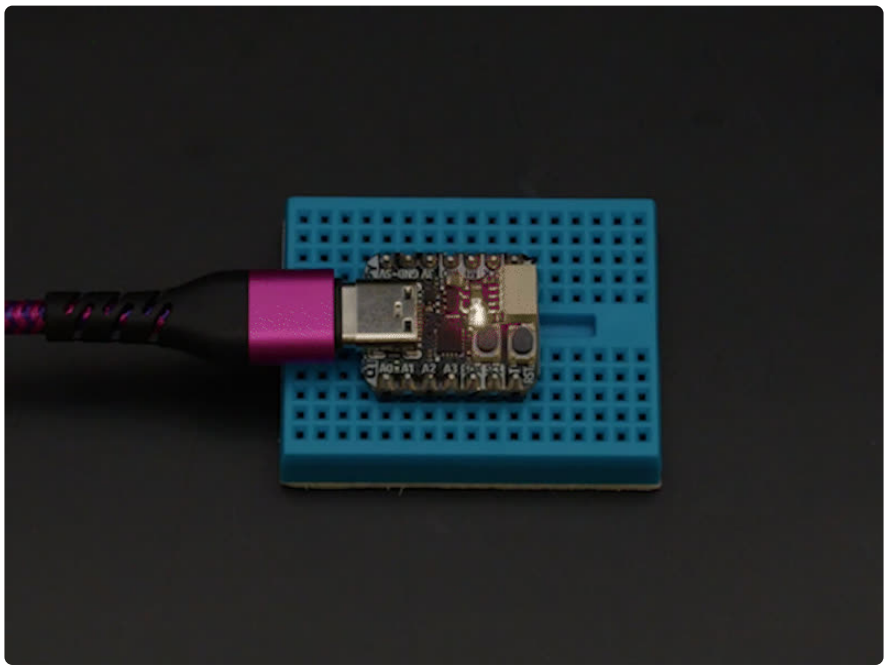


Now upload the scanning sketch to your microcontroller and open the serial port to see the output. You should see something like this:



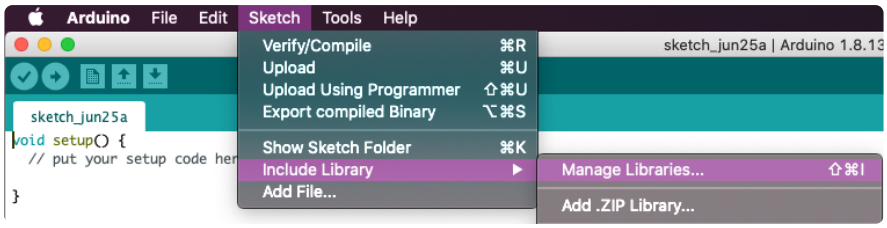
NeoPixel

In this example, you'll upload the sketch to have the onboard NeoPixel (pin PA4) perform a rainbow swirl.

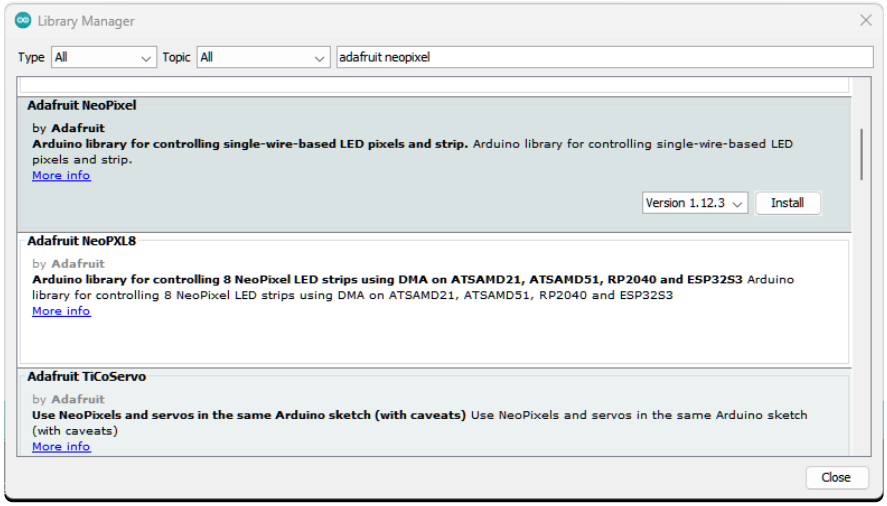


Library Installation

You can install the **Adafruit_NeoPixel** library for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit_NeoPixel**, and select the **Adafruit NeoPixel** library:



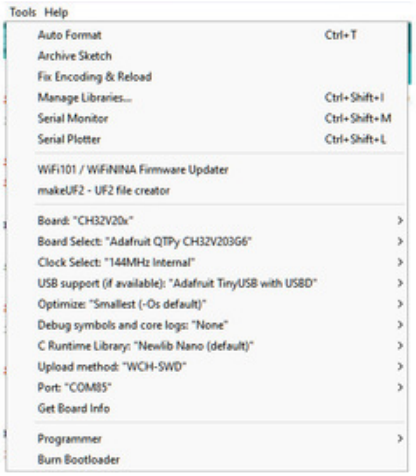
There are no additional dependencies for this library.

NeoPixel Example

```
// SPDX-FileCopyrightText: 2024 ladyada for Adafruit Industries
//
// SPDX-License-Identifier: MIT

#include <Arduino.h>
#include <Adafruit_TinyUSB.h> // required for USB Serial
#include <Adafruit_NeoPixel.h>

const int neopixel_pin = PA4;
#define LED_COUNT 1
Adafruit_NeoPixel pixels(LED_COUNT, neopixel_pin, NEO_GRB + NEO_KHZ800);

void setup () {
  pixels.begin();
}

uint16_t firstPixelHue = 0;

void loop() {
  firstPixelHue += 256;
  for(int i=0; i<pixels.numPixels(); i++) {
      int pixelHue = firstPixelHue + (i * 65536L / pixels.numPixels());
      pixels.setPixelColor(i, pixels.gamma32(pixels.ColorHSV(pixelHue)));
    }
```

```
  pixels.show();
  delay(10);
}
```

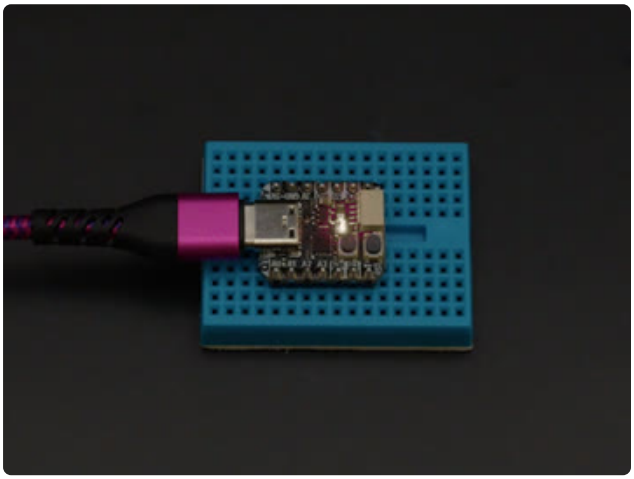
You'll notice that at the top of the code that the TinyUSB library is imported with `#include <Adafruit_TinyUSB.h>`. This will allow access to the USB serial port on the QT Py.



Confirm that your upload settings match the settings listed here under **Tools**:

Board: **CH32V20x**
Board Select: **Adafruit QTPy CH32V203G6**
USB support: **Adafruit TinyUSB with USBD**

For the port, select the COM port that matches your QT Py. It will not be labeled like you may be used to with other boards in the Arduino IDE.

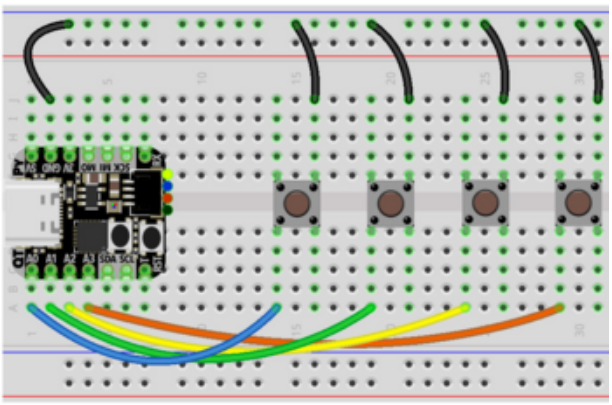


Upload the sketch to your board. You'll see the NeoPixel begin swirling thru the colors of the rainbow. This is the same demo that ships on the boards.

HID Keyboard

In this example, you'll upload the sketch to turn your QT Py into a simple four key keyboard.

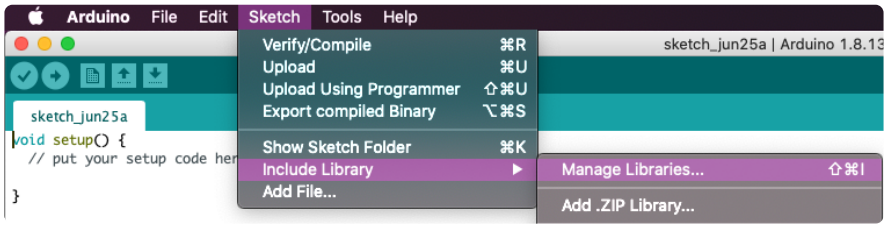
Wiring



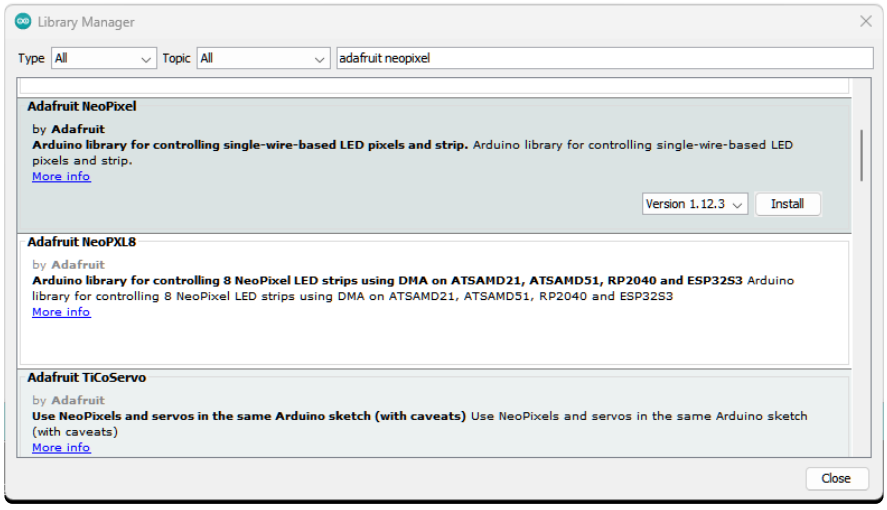
Button 1 input to **board A0 (blue wire)**
Button 1 ground to **board GND (black wire)**
Button 2 input to **board A1 (green wire)**
Button 2 ground to **board GND (black wire)**
Button 3 input to **board A2 (yellow wire)**
Button 3 ground to **board GND (black wire)**
Button 4 input to **board A3 (orange wire)**
Button 4 ground to **board GND (black wire)**

Library Installation

You can install the **Adafruit_NeoPixel** library for Arduino using the Library Manager in the Arduino IDE.



Click the **Manage Libraries ...** menu item, search for **Adafruit_NeoPixel**, and select the **Adafruit NeoPixel** library:



There are no additional dependencies for this library.

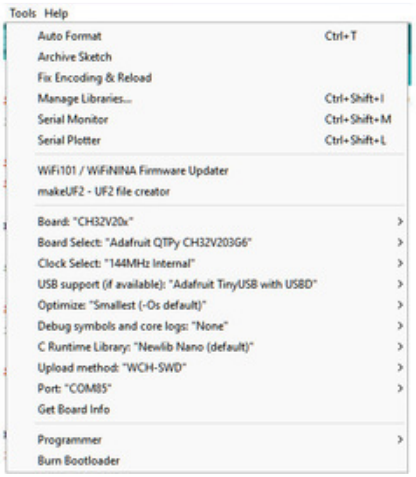
This example also requires the Adafruit TinyUSB library. You should already have this library installed after following the steps on the Arduino IDE (https://adafru.it/1a6T) setup page.

HID Keyboard Example

```
// SPDX-FileCopyrightText: 2024 Ha Thach for Adafruit Industries
//
// SPDX-License-Identifier: MIT

/*********************************************************************
 Adafruit invests time and resources providing this open source code,
 please support Adafruit and open-source hardware by purchasing
 products from Adafruit!

 MIT license, check LICENSE for more information
 Copyright (c) 2019 Ha Thach for Adafruit Industries
 All text above, and the splash screen below must be included in
 any redistribution
*********************************************************************/

#include <Arduino.h>
#include "Adafruit_TinyUSB.h"
#include <Adafruit_NeoPixel.h>

const int neopixel_pin = PA4;
#define LED_COUNT 1
Adafruit_NeoPixel pixels(LED_COUNT, neopixel_pin, NEO_GRB + NEO_KHZ800);

//------------- Input Pins -------------//
// Array of pins and its keycode.
// Notes: these pins can be replaced by PIN_BUTTONn if defined in setup()

uint8_t pins[] = {PB1, PB0, PA1, PA0}; //A0, A1, A2, A3

// HID report descriptor using TinyUSB's template
// Single Report (no ID) descriptor
uint8_t const desc_hid_report[] = {
    TUD_HID_REPORT_DESC_KEYBOARD()
};

// USB HID object. For ESP32 these values cannot be changed after this declaration
// desc report, desc len, protocol, interval, use out endpoint
Adafruit_USBD_HID usb_hid;


// number of pins
uint8_t pincount = sizeof(pins) / sizeof(pins[0]);

// For keycode definition check out https://github.com/hathach/tinyusb/blob/master/
src/class/hid/hid.h
uint8_t hidcode[] = {HID_KEY_0, HID_KEY_1, HID_KEY_2, HID_KEY_3};

bool activeState = false;

// the setup function runs once when you press reset or power the board
void setup() {
  pixels.begin();
  // Manual begin() is required on core without built-in support e.g. mbed rp2040
```

```
  if (!TinyUSBDevice.isInitialized()) {
    TinyUSBDevice.begin(0);
  }

  // Setup HID
  usb_hid.setBootProtocol(HID_ITF_PROTOCOL_KEYBOARD);
  usb_hid.setPollInterval(2);
  usb_hid.setReportDescriptor(desc_hid_report, sizeof(desc_hid_report));
  usb_hid.setStringDescriptor("TinyUSB Keyboard");

  // Set up output report (on control endpoint) for Capslock indicator
  usb_hid.setReportCallback(NULL, hid_report_callback);

  usb_hid.begin();

  // overwrite input pin with PIN_BUTTONx
#ifdef PIN_BUTTON1
  pins[0] = PIN_BUTTON1;
#endif

#ifdef PIN_BUTTON2
  pins[1] = PIN_BUTTON2;
#endif

#ifdef PIN_BUTTON3
  pins[2] = PIN_BUTTON3;
#endif

#ifdef PIN_BUTTON4
  pins[3] = PIN_BUTTON4;
#endif

  // Set up pin as input
  for (uint8_t i = 0; i < pincount; i++) {
    pinMode(pins[i], activeState ? INPUT_PULLDOWN : INPUT_PULLUP);
  }
}

void process_hid() {
  // used to avoid send multiple consecutive zero report for keyboard
  static bool keyPressedPreviously = false;

  uint8_t count = 0;
  uint8_t keycode[6] = {0};

  // scan normal key and send report
  for (uint8_t i = 0; i < pincount; i++) {
    if (activeState == digitalRead(pins[i])) {
      // if pin is active (low), add its hid code to key report
      keycode[count++] = hidcode[i];

      // 6 is max keycode per report
      if (count == 6) break;
    }
  }

  if (TinyUSBDevice.suspended() && count) {
    // Wake up host if we are in suspend mode
    // and REMOTE_WAKEUP feature is enabled by host
    TinyUSBDevice.remoteWakeup();
  }

  // skip if hid is not ready e.g still transferring previous report
  if (!usb_hid.ready()) return;

  if (count) {
    // Send report if there is key pressed
    uint8_t const report_id = 0;
    uint8_t const modifier = 0;
```

```
    keyPressedPreviously = true;
    usb_hid.keyboardReport(report_id, modifier, keycode);
  } else {
    // Send All-zero report to indicate there is no keys pressed
    // Most of the time, it is, though we don't need to send zero report
    // every loop(), only a key is pressed in previous loop()
    if (keyPressedPreviously) {
      keyPressedPreviously = false;
      usb_hid.keyboardRelease(0);
    }
  }
}

void loop() {
  #ifdef TINYUSB_NEED_POLLING_TASK
  // Manual call tud_task since it isn't called by Core's background
  TinyUSBDevice.task();
  #endif

  // not enumerated()/mounted() yet: nothing to do
  if (!TinyUSBDevice.mounted()) {
    return;
  }

  // poll gpio once each 2 ms
  static uint32_t ms = 0;
  if (millis() - ms > 2) {
    ms = millis();
    process_hid();
  }
}

void setLED(bool state) {
  pixels.setPixelColor(0, pixels.Color(0, state ? 150 : 0, 0));
  pixels.show();
}

// Output report callback for LED indicator such as Caplocks
void hid_report_callback(uint8_t report_id, hid_report_type_t report_type, uint8_t
const* buffer, uint16_t bufsize) {
  (void) report_id;
  (void) bufsize;

  // LED indicator is output report with only 1 byte length
  if (report_type != HID_REPORT_TYPE_OUTPUT) return;

  // The LED bit map is as follows: (also defined by KEYBOARD_LED_* )
  // Kana (4) | Compose (3) | ScrollLock (2) | CapsLock (1) | Numlock (0)
  uint8_t ledIndicator = buffer[0];

  // turn on LED if capslock is set
  setLED(ledIndicator & KEYBOARD_LED_CAPSLOCK);
}
```



Confirm that your upload settings match the settings listed here under **Tools**:

Board: **CH32V20x**
Board Select: **Adafruit QTPy CH32V203G6**
USB support: **Adafruit TinyUSB with USBD**

For the port, select the COM port that matches your QT Py. It will not be labeled like you may be used to with other boards in the Arduino IDE.

Upload the sketch to your board. When you press the buttons connected to pins A0-A3, you'll type 0, 1, 2 or 3.

You can customize the code to send different keycodes. Edit the `hidcode[]` with the keycodes of your choice.

```
// For keycode definition check out https://github.com/hathach/tinyusb/blob/master/
src/class/hid/hid.h
uint8_t hidcode[] = {HID_KEY_0, HID_KEY_1, HID_KEY_2, HID_KEY_3};
```

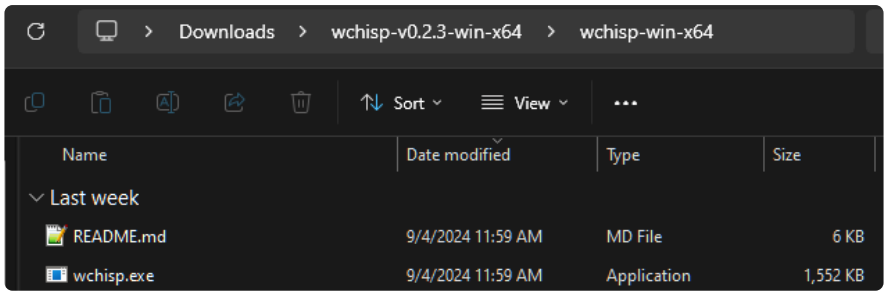
Using WCHISP Tool

Uploading directly to the QT Py USB port with the Arduino IDE is very convenient. However, if you find that you can't access the COM port or have problems with your toolchain then you may need to utilize uploading code in bootloader mode with the WCHISP tool (https://adafru.it/1a6U). The WCHISP tool is a command line implementation of the WCHISP tool in Rust.

You can get your QT Py CH32V203 into bootloader mode by unplugging the USB port, holding down the **Boot** button and plugging the USB cable back in.

Install WCHISP Tool

There are prebuilt binaries of the WCHISP tool on the GitHub repository. Navigate to the releases page (https://adafru.it/1a6V) and download the binary for your operating system. After installing, the tool will be available in a folder:



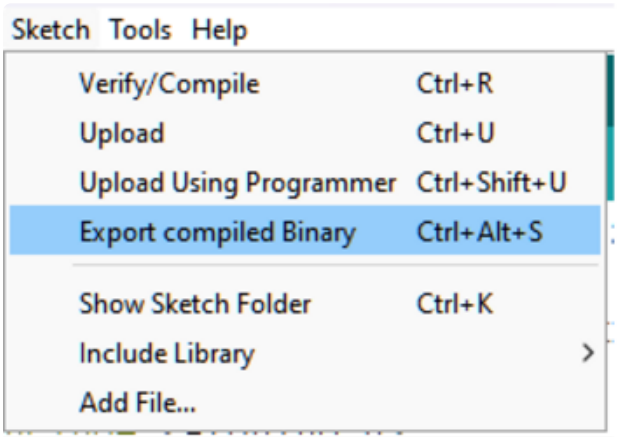
Using WCHISP Tool

After you install the tool, navigate to its directory in your terminal:

```
cd /path/to/wchisp
```

Place the QT Py into bootloader mode by holding down the BOOT button and plugging it in via USB. To confirm that your board is connected, you can run `wchisp` in the terminal. You should see that the tool has found the device:

```
C:\Users\lizillah\Downloads\wchisp-v0.2.3-win-x64\wchisp-win-x64>wchisp
14:26:04 [INFO] Found 1 devices
14:26:04 [INFO] hint: use `wchisp info` to check chip info
Device #0: CH32V203G6U6[0x3619]
```



To flash code onto the board, you'll need a compiled binary file. These files can be compiled in the Arduino IDE by going to **Sketch - Export compiled Binary**. The binary file will be exported to the Sketch folder. Make sure to use the same board settings (https://adafru.it/1a6T) that are described on the Arduino IDE pages in this guide.

Compiled binary files are available for the examples in this guide (https://adafru.it/1a6W) in the Learn repository. You can also use the Factory Reset binary (https://adafru.it/1a6X), which is the NeoPixel swirl example that the boards ship with.

Factory Reset Binary File

https://adafru.it/1a6Y

To upload the code to the board, run the following command in the terminal:

```
wchisp flash /path/to/binary_file.bin
```

You should see an output like this in the terminal after the binary has been successfully uploaded:

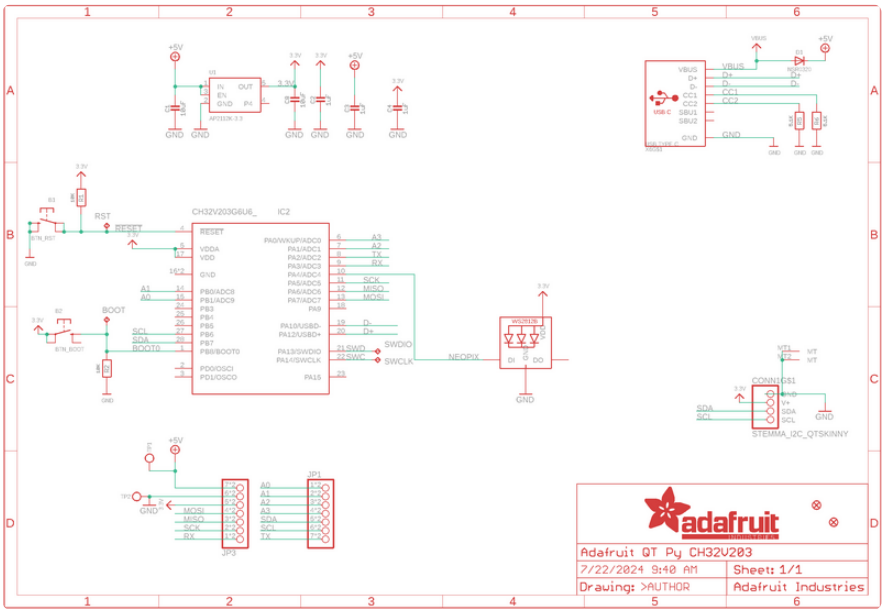
```
C:\Users\lizillah\Downloads\wchisp-v0.2.3-win-x64\wchisp-win-x64>wchisp flash binary_file.bin
14:33:24 [INFO] Chip: CH32V203G6U6[0x3619] (Code Flash: 32KiB)
14:33:24 [INFO] Chip UID: CD-AB-5B-5C-13-BC-3B-C4
14:33:24 [INFO] BTVER(bootloader ver): 02.70
14:33:24 [INFO] Code Flash protected: false
14:33:24 [INFO] Current config registers: a55a3fc000ff00ffffffffff00020700cdab5b5c13bc3bc4
RDPR_USER: 0xC03F5AA5
  [7:0]   RDPR 0xA5 (0b10100101)
    `- Unprotected
  [16:16] IWDG_SW 0x1 (0b1)
    `- IWDG enabled by the software, and disabled by hardware
  [17:17] STOP_RST 0x1 (0b1)
    `- Disable
  [18:18] STANDBY_RST 0x1 (0b1)
    `- Disable, entering standby-mode without RST
  [23:22] SRAM_CODE_MODE 0x0 (0b0)
    `- CODE-192KB + RAM-128KB / CODE-128KB + RAM-64KB depending on the chip
DATA: 0xFF00FF00
  [7:0]   DATA0 0x0 (0b0)
  [23:16] DATA1 0x0 (0b0)
WRP: 0xFFFFFFFF
  `- Unprotected
14:33:24 [INFO] Read binary_file.bin as Binary format
14:33:24 [INFO] Firmware size: 30720
14:33:24 [INFO] Erasing...
14:33:24 [INFO] Erased 31 code flash sectors
14:33:25 [INFO] Erase done
14:33:25 [INFO] Writing to code flash...
                                                                                30720/30720
14:33:25 [INFO] Code flash 30720 bytes written
14:33:26 [INFO] Verifying...
                                                                                30720/30720
14:33:26 [INFO] Verify OK
14:33:26 [INFO] Now reset device and skip any communication errors
14:33:26 [INFO] Device reset
```

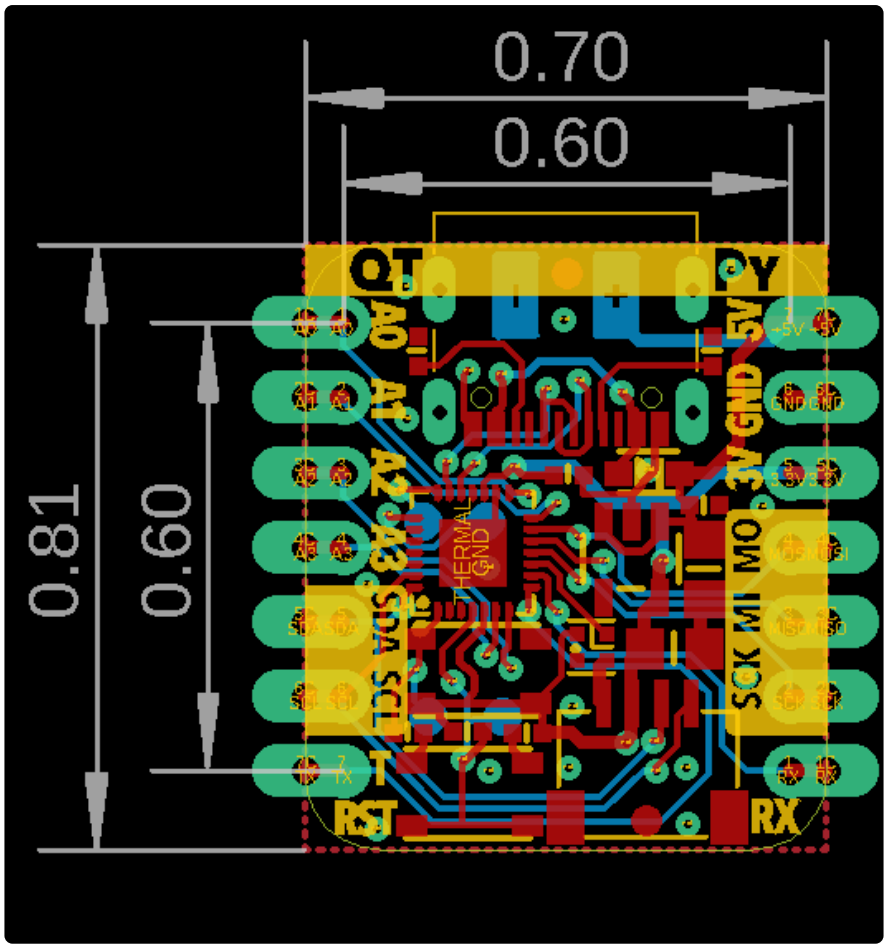
Downloads

Files

- CH32V203 Product Page (https://adafru.it/1a6Z)
- CH32V203 Datasheet (https://adafru.it/1a70)
- EagleCAD PCB files on GitHub (https://adafru.it/1a71)
- Fritzing object in the Adafruit Fritzing Library (https://adafru.it/1a72)
- PrettyPins pinout PDF on GitHub (https://adafru.it/1a6P)
- PrettyPins pinout SVG on GitHub (https://adafru.it/1a73)

Schematic and Fab Print




0.70
0.60
0.81
0.60
QT
PY
A0
A1
A2
A3
SDA
SCL
5V
GND
3V
MO
MI
SCK
RX
THERMAL GND